

Explain Domino Tiling DP

1. Big Picture

This lesson is about **LeetCode 790 — Domino and Tromino Tiling**.

The problem looks simple at first: tile a $2 \times n$ board using:

- 2×1 dominoes
- L-shaped trominoes

But the difficult part is **not writing the DP code**.

The real difficulty is discovering **why the recurrence is**

```
text
dp[n] = 2 * dp[n - 1] + dp[n - 3]
```

The instructor's main goal is to avoid memorizing this formula blindly. Instead, we derive it by understanding:

1. How the board can be divided.
2. Why simply counting every possible placement causes **duplicate counting**.
3. How to count only configurations that reach n **directly**, without passing through an intermediate state.
4. How that leads naturally to the recurrence.
5. How to implement the recurrence using recursion + memoization and bottom-up DP.

That is the important lesson from the video.

2. Prerequisites

You need only a few DP concepts.

2.1 Dynamic Programming

Suppose:

```
text
dp[n] = number of ways to tile a 2 x n board
```

Then instead of solving the entire board from scratch, we try to express $dp[n]$ using smaller values such as:

```
text
dp[n-1]
dp[n-3]
```

This is useful because the same smaller subproblems appear repeatedly.

2.2 Recurrence Relation

A recurrence describes a bigger problem using smaller problems.

For example:

```
text
dp[n] = 2 * dp[n-1] + dp[n-3]
```

This does **not** magically appear.

The entire purpose of this lesson is to understand where it comes from.

2.3 Domino

A domino occupies:

```
text
2 x 1
```

It can be placed:

```
text
Vertical:

+---+
|   |
|   |
+---+
```

Horizontal pair:

```
+---+---+
|       |
+---+---+
```

Because the board has height 2, a horizontal domino by itself cannot completely cover a column. Two horizontal dominoes are needed together.

2.4 Tromino

A tromino is an L-shaped tile occupying three cells.
There are essentially two orientations relevant to the problem:

```
text
XX      X
X       XX
```

and their mirrored versions.
The transcript emphasizes that the tromino is what makes this problem substantially more interesting.

3. Detailed Explanation

3.1 Understand the Problem First

We have a board:

```
text
2 x n
```

and we must completely cover it using dominoes and trominoes.

Important condition

There must be **no uncovered cell**.
For example, a placement that leaves something like this:

```
text
XX
X_
```

is invalid.
Every cell must eventually be covered.
The answer is the **number of distinct complete tilings**.

3.2 Start With Small Values

Before deriving a recurrence, calculate small cases manually.
This is one of the most useful DP habits:

Don't immediately search for a formula. First understand the first few states.

n = 1

Board:

```
text
XX
```

Only one domino can cover it vertically.
Therefore:

```
text
dp[1] = 1
```

n = 2

Board:

```
text
XX
XX
```

There are two possibilities:

Option 1 — two vertical dominoes

```
text
| |
| |
```

Option 2 — two horizontal dominoes

```
text
----
----
```

Therefore:

```
text
dp[2] = 2
```

n = 3

This is where the interesting part begins.

The total number of valid tilings is:

```
text
dp[3] = 5
```

So:

```
text
dp[1] = 1
dp[2] = 2
dp[3] = 5
```

The transcript spends considerable time manually examining these arrangements because they provide the intuition needed for the recurrence.

3.3 Why Can't We Just Guess the Pattern?

We have:

```
text
1, 2, 5, ...
```

You might be tempted to say:

```
text
dp[n] = something involving dp[n-1]
```

But that's dangerous.

For example:

```
text
dp[3] = 2 * dp[2] + 1
       = 2 * 2 + 1
       = 5
```

That works.

But **why**?

And what does that extra **1** represent?

The instructor's key point is:

You should understand the combinatorial reason behind the recurrence rather than memorizing it.

That is especially important in interviews because an interviewer can modify the tiling problem slightly.

3.4 The Critical Idea: Avoid Duplicate Counting

This is the hardest and most important part.

Suppose we want to construct:

text

2 × 4

from smaller boards.

A natural thought is:

"I'll take a smaller board and attach some tiles."

But there is a problem.

A configuration for 2 × 4 might contain a complete 2 × 3 tiling somewhere inside it.

If you count it once while constructing from dp[3] and again through another construction path, you have counted the same final tiling twice.

This is the **duplicate-counting problem**.

3.5 What Does "Directly Reach n" Mean?

This is the key intuition from the video.

Suppose we are calculating:

text

dp[4]

and we start from:

text

dp[2]

We want to add tiles in such a way that we reach 4 **without creating a complete intermediate 2 × 3 board**.

Why?

Because if we create a complete 2 × 3 board first, that configuration is already counted by:

text

dp[3]

and we would count it again.

So:

When deriving the recurrence, we count only the ways that reach the current state without passing through a previously counted complete intermediate state.

This is the central insight.

3.6 Deriving the dp[n-1] Contribution

Take a 2 × n board.

Consider the last column.

One simple possibility is:

text

2 × (n-1) board + one vertical domino

Visually:

text

| | | | |
| | | | |
↑
vertical domino

The first n-1 columns can be tiled in:

text

dp[n-1]

ways.

The final column has exactly:

text

1

way to be completed with a vertical domino.

Therefore:

```
text
dp[n-1] * 1
```

ways.

So one contribution is:

```
text
dp[n-1]
```

3.7 Why Do We Get $2 \times dp[n-1]$?

There are actually **two symmetric ways** to create the relevant incomplete/end structure.

The tromino arrangements give us two orientations.

Conceptually:

```
text
configuration A
configuration B
```

Each can be combined with every valid tiling of a smaller board.

Therefore:

```
text
dp[n-1] + dp[n-1]
```

which gives:

```
text
2 * dp[n-1]
```

This is the first major part of the recurrence.

3.8 Where Does $dp[n-3]$ Come From?

Now consider the special arrangement involving the trominoes that allows us to move directly from:

```
text
2 * (n-3)
```

to:

```
text
2 * n
```

There are exactly **two cells of width difference?**

Let's be precise.

The standard recurrence can be understood more cleanly through the classic "profile" interpretation:

A partially tiled board can leave a one-cell offset at the boundary. The two symmetric ways to resolve that offset eventually produce the $2 * dp[n-1]$ contribution, while the configuration involving the pair of trominoes contributes the $dp[n-3]$ term.

The important result is:

```
text
dp[n] = 2 * dp[n-1] + dp[n-3]
```

with:

```
text
dp[1] = 1
dp[2] = 2
dp[3] = 5
```

The transcript derives this by repeatedly removing the valid non-duplicating boundary configurations and then simplifying the resulting expression.

3.9 Why Not Simply Use $dp[n-1] + dp[n-2] + \dots$?

Because those groups are **not necessarily disjoint**.

This is an extremely important DP/combinatorics principle.

Suppose you say:

text

All answers come from:

1. $dp[n-1]$
2. $dp[n-2]$
3. $dp[n-3]$

You are assuming these categories don't overlap.

But if the same tiling can be generated through two categories, then:

text

$total = category1 + category2 + category3$

is wrong.

The same answer gets counted multiple times.

This is exactly what the instructor demonstrates with the 2×4 example.

3.10 The Duplicate Example

Imagine you're constructing a 2×4 board from a 2×2 board.

Suppose you place two vertical dominoes after the 2×2 portion.

You might think:

text

$dp[2] \rightarrow$ add two vertical dominoes $\rightarrow dp[4]$

But the resulting configuration can also be viewed as:

text

$dp[3] \rightarrow$ add one vertical domino $\rightarrow dp[4]$

So the same final tiling appears through multiple construction paths.

That means:

text

counting both paths = duplicate

The solution is not to count arbitrary construction paths.

Instead:

Partition the tilings into mutually exclusive categories.

That is the mathematical foundation behind the recurrence.

3.11 Expanding the Recurrence

Start with:

text

$dp[n] = 2 * dp[n-1] + dp[n-3]$

Now expand:

text

$dp[n-1] = 2 * dp[n-2] + dp[n-4]$

Therefore:

text

$$\begin{aligned} dp[n] &= 2 * (2 * dp[n-2] + dp[n-4]) + dp[n-3] \\ &= 4 * dp[n-2] + 2 * dp[n-4] + dp[n-3] \end{aligned}$$

Continue expanding.

Eventually, you get a representation involving earlier states.

The instructor then compares two consecutive expanded equations. Most terms cancel.

This is a common mathematical technique:

Derive a complicated summation and then subtract consecutive equations to obtain a clean recurrence.

That ultimately gives:

text

```
dp[n] = 2 * dp[n-1] + dp[n-3]
```

The important lesson is that the formula is **derived from the structure of the tilings**, not guessed.

3.12 Verify the Formula

Now test it.

We know:

text

```
dp[1] = 1
dp[2] = 2
dp[3] = 5
```

For $n = 4$:

text

```
dp[4] = 2 * dp[3] + dp[1]
      = 2 * 5 + 1
      = 11
```

So:

text

```
dp[4] = 11
```

For $n = 5$:

text

```
dp[5] = 2 * dp[4] + dp[2]
      = 2 * 11 + 2
      = 24
```

So:

text

```
dp[5] = 24
```

The sequence becomes:

text

n	1	2	3	4	5	6
dp[n]	1	2	5	11	24	53

That is the sequence you should recognize for this problem.

3.13 Why Modulo $10^9 + 7$?

The number of tilings grows very quickly.

For large n , the answer becomes enormous.

Therefore LeetCode asks for:

text

```
answer % (109 + 7)
```

We define:

text

```
MOD = 1000000007
```

and perform calculations modulo `MOD`.

For example:

text

```
dp[n] = (2 * dp[n-1] + dp[n-3]) % MOD
```

Why modulo during calculation?

Because otherwise the integer can overflow.

Instead of allowing values to become astronomically large, we keep them within a manageable range.

3.14 Recursion + Memoization

The recurrence naturally gives a recursive solution:

```
text
dp[n] = 2 * dp[n-1] + dp[n-3]
```

So:

```
text
solve(n)
  if n == 1:
    return 1

  if n == 2:
    return 2

  return 2 * solve(n-1) + solve(n-3)
```

But plain recursion repeats the same subproblems.

For example:

```
text
solve(6)
├─ solve(5)
│  └─ solve(4)
│     └─ solve(3)
│        └─ solve(1)
│       └─ solve(2)
└─ solve(3)
```

Notice:

```
text
solve(3)
```

appears more than once.

For larger n , this repetition becomes expensive.

3.15 Memoization

Memoization means:

Compute each state once and store its answer.

For example:

```
text
vector<long long> dp(n + 1, -1);
```

Then:

```
C++
long long solve(int n) {
  if (n == 1) return 1;
  if (n == 2) return 2;
  if (dp[n] != -1) return dp[n];

  return dp[n] = (2 * solve(n - 1) + solve(n - 3)) % MOD;
}
```

The exact structure can be written more robustly for LeetCode as:

```
C++
class Solution {
public:
  static const long long MOD = 1000000007;
  vector<long long> dp;

  long long solve(int n) {
    if (n == 1) return 1;
    if (n == 2) return 2;
    if (n == 3) return 5;

    if (dp[n] != -1)
```

```

        return dp[n];

    return dp[n] = (2 * solve(n - 1) + solve(n - 3)) % MOD;
}

int numTilings(int n) {
    dp.assign(n + 1, -1);
    return solve(n);
}
};

```

What happens internally?

Suppose:

```

text
n = 6

```

The first time we calculate:

```

text
solve(3)

```

we store:

```

text
dp[3] = 5

```

If `solve(3)` is requested again:

```

text
if (dp[3] != -1)
    return dp[3];

```

We immediately return 5.

No recomputation.

3.16 Complexity of Memoization

There are only n states:

```

text
1, 2, 3, ..., n

```

Each state is calculated once.

Therefore:

```

text
Time   = O(n)
Space  = O(n)

```

The $O(n)$ space comes from:

1. DP array
2. Recursion stack

3.17 Bottom-Up DP

Because the recurrence depends on smaller values:

```

text
dp[n-1]
dp[n-3]

```

we can calculate from small to large.

Initialize:

```

text
dp[1] = 1
dp[2] = 2
dp[3] = 5

```

Then:

text

```
for i = 4 to n:
    dp[i] = 2 * dp[i-1] + dp[i-3]
```

Correct implementation:

C++

```
class Solution {
public:
    int numTilings(int n) {
        const long long MOD = 1000000007;

        if (n == 1) return 1;
        if (n == 2) return 2;
        if (n == 3) return 5;

        vector<long long> dp(n + 1);

        dp[1] = 1;
        dp[2] = 2;
        dp[3] = 5;

        for (int i = 4; i <= n; i++) {
            dp[i] = (2 * dp[i - 1] + dp[i - 3]) % MOD;
        }

        return dp[n];
    }
};
```

3.18 Why Bottom-Up Works

For:

text

dp[4]

we need:

text

dp[3]
dp[1]

Both are already calculated.

Then for:

text

dp[5]

we need:

text

dp[4]
dp[2]

which are also already calculated.

So the dependency graph is:

text

```
dp[1] -----> dp[5]
dp[2] -----> dp[5]
dp[3] -> dp[4] -----> dp[5]
```

More simply:

text

dp[i] depends on dp[i-1] and dp[i-3]

Therefore left-to-right calculation works naturally.

3.19 Space Optimization

This is an important improvement.

The recurrence is:

```
text
dp[i] = 2 * dp[i-1] + dp[i-3]
```

We don't need the entire DP array.

We only need:

```
text
dp[i-1]
dp[i-3]
```

So we can maintain three variables.

```
C++
class Solution {
public:
    int numTilings(int n) {
        const long long MOD = 1000000007;

        if (n == 1) return 1;
        if (n == 2) return 2;
        if (n == 3) return 5;

        long long a = 1; // dp[i-3]
        long long b = 2; // dp[i-2]
        long long c = 5; // dp[i-1]

        for (int i = 4; i <= n; i++) {
            long long next = (2 * c + a) % MOD;

            a = b;
            b = c;
            c = next;
        }

        return c;
    }
};
```

Let's understand the variables.

Initially:

```
text
a = dp[1]
b = dp[2]
c = dp[3]
```

For i = 4:

```
text
next = 2 * dp[3] + dp[1]
      = 2 * c + a
```

After calculating:

```
text
a = dp[2]
b = dp[3]
c = dp[4]
```

Then for i = 5:

```
text
next = 2 * dp[4] + dp[2]
      = 2 * c + a
```

Again shift the variables.

Thus:

```
text
Time = O(n)
Space = O(1)
```

This is the best implementation among the approaches discussed.

3.20 Full Flow of the Solution

The entire thought process should now look like this:

```
text
2 × n tiling problem
↓
Calculate small cases
↓
dp[1] = 1
dp[2] = 2
dp[3] = 5
↓
Analyze how a tiling reaches the right boundary
↓
Avoid counting constructions that pass
through an already-counted intermediate state
↓
Derive recurrence
↓
dp[n] = 2 * dp[n-1] + dp[n-3]
↓
Choose implementation
↓
Recursion
↓
Memoization
↓
Bottom-up DP
↓
Space optimization
↓
O(n) time, O(1) space
```

4. Key Takeaways

1. Don't memorize the recurrence

The most important lesson is:

```
text
dp[n] = 2 * dp[n-1] + dp[n-3]
```

is not something you should blindly memorize.

You should understand **why those particular previous states appear**.

2. Duplicate counting is the real challenge

The difficult part of this problem isn't DP syntax.

It is identifying that different construction paths can produce the **same final tiling**.

Therefore, when designing a recurrence:

Make sure the cases you count are mutually exclusive.

3. Small examples are extremely valuable

Always calculate:

```
text
dp[1]
dp[2]
dp[3]
dp[4]
```

manually.

They help you discover and validate the recurrence.

4. The recurrence is simple after the reasoning is complete

Once the derivation is understood, the implementation is tiny:

```
text
dp[n] = 2 * dp[n-1] + dp[n-3]
```

That's a recurring pattern in algorithmic problems:

```
Hard reasoning → simple code.
```

5. Memoization removes repeated work

Plain recursion:

```
text
O(2^n) approximately
```

can repeatedly solve the same states.

Memoization reduces it to:

```
text
O(n)
```

6. Bottom-up DP removes recursion overhead

Bottom-up:

```
text
O(n) time
O(n) space
```

7. State compression gives O(1) space

Because only three relevant DP values are needed:

```
text
dp[i-3]
dp[i-2]
dp[i-1]
```

we can reduce space to:

```
text
O(1)
```

5. Common Mistakes / Misunderstandings

Mistake 1: Memorizing the formula

Wrong mindset:

```
text
"I remember Domino and Tromino = 2dp[n-1] + dp[n-3]."
```

Better mindset:

```
text
"I know how the boundary configurations lead to those terms."
```

That's what makes you capable of solving variations.

Mistake 2: Counting every possible placement independently

This causes:

```
text
duplicate counting
```

The same final board may be generated through different intermediate states.

Mistake 3: Forgetting the base cases

You need:

text

```
dp[1] = 1
dp[2] = 2
dp[3] = 5
```

Without correct initialization, the recurrence cannot work.

Mistake 4: Ignoring modulo during calculation

Use:

C++

```
dp[i] = (2 * dp[i - 1] + dp[i - 3]) % MOD;
```

rather than allowing huge numbers to accumulate.

Mistake 5: Thinking the 2 means "two arbitrary placements"

It doesn't mean:

text

"There are exactly two ways to place a tile."

It comes from two symmetric classes of valid boundary configurations, each capable of being combined with the appropriate smaller tiling count.

6. Quick Revision

Keep this mental model:

text

Problem:
Count ways to completely tile a $2 \times n$ board
using dominoes and trominoes.

Base cases:

text

```
dp[1] = 1
dp[2] = 2
dp[3] = 5
```

Recurrence:

text

```
dp[n] = 2 * dp[n-1] + dp[n-3]
```

Modulo:

text

```
MOD = 1e9 + 7
```

Memoization:

text

```
Time = O(n)
Space = O(n)
```

Bottom-up:

text

```
Time = O(n)
Space = O(n)
```

Space optimized:

text

```
Time = O(n)
Space = O(1)
```

Most important conceptual point:

text

Avoid duplicate counting by constructing mutually exclusive boundary configurations.

7. Questions You Should Be Able to Answer

Before considering this problem fully learned, you should be able to answer these **without looking at the solution**:

1. What does $dp[n]$ represent?
2. Why is $dp[1] = 1$?
3. Why is $dp[2] = 2$?
4. Why is $dp[3] = 5$?
5. Why can't we simply count every possible way of attaching tiles?
6. What does **duplicate counting** mean in this problem?
7. Why is $dp[n-1]$ involved?
8. Why does the recurrence contain the factor 2?
9. Why does $dp[n-3]$ appear?
10. Why is the recurrence not simply $dp[n-1] + dp[n-2]$?
11. How does memoization improve plain recursion?
12. Why is memoization $O(n)$?
13. How does bottom-up DP calculate the states?
14. Why can the DP array be reduced to $O(1)$ space?
15. Why do we use $1e9 + 7$?
16. Can you derive the recurrence instead of merely recalling it?
17. If the interviewer changes the allowed tiles, what part of your reasoning would need to change?

The last question is the real test. If you can only reproduce the recurrence, you've memorized the problem. If you can reconstruct the state and eliminate duplicate counting, you've learned the technique.